

Key Java Rules to Remember

1. **Classes cannot extend multiple classes** – avoids state/method ambiguity.
 2. **Interfaces can extend multiple interfaces**, even with default methods.
 3. **Conflict only arises with multiple default methods** (or default vs abstract).
 4. **You must explicitly override** when a conflict occurs and resolve it using
`InterfaceName.super.method()`.
-

Real-World Strategy

- Use **interfaces** for multiple inheritance of behavior.
- Prefer **default methods** in interfaces for reusable behavior.
- When mixing interfaces, be mindful of **method name clashes**.
- Always resolve ambiguity **explicitly** if it arises.

Java Interface Inheritance: Concepts and Scenarios

1. Basic Interface Inheritance

When one interface extends another, **it inherits all of its methods** — including:

- Abstract methods
 - Default methods
 - Static methods (though not inherited **as-is** — more on this later)
-

Example

```
interface A {
    void method(); // abstract
}

interface B extends A {
    // No need to declare method() again - it's inherited
}
```

- B **inherits** `method()` from A.
- You don't need to redeclare it unless you're changing its behavior or purpose.

◆ 2. When is Redeclaring a Method in a Child Interface Optional, Required, or Useful?

Purpose	Redeclaration in B	Explanation
✓ Inherit abstract method	✗ Not required	It's inherited automatically
✓ Add <code>default</code> implementation	✓ Required	Must be declared with <code>default</code> to provide body
✓ Add <code>static</code> method	✓ Required	Must redeclare as <code>static</code> with body
✓ Override a default method	✓ Required	You must redeclare and override
✎ Add JavaDoc / comments	✓ Optional	Redeclaration can add documentation or intent clarity

◆ 2.1. Redeclaring an Abstract Method – Optional

```
interface A {  
    void method(); // abstract  
}  
  
interface B extends A {  
    void method(); // ✓ Legal but redundant  
}
```

- This is **not necessary**, but allowed.
 - Java treats it as the same abstract method — there's no new behavior.
-

◆ 2.2. Providing a `default` Implementation – Required

```
interface A {  
    void method();  
}  
  
interface B extends A {  
    @Override  
    default void method() {  
        System.out.println("Default method in B");  
    }  
}
```

- ✓ Now B provides a **default implementation** of `method()`.
- Any interface/class extending or implementing B will inherit this default unless overridden.

◆ 2.3. Overriding a Default Method in a Subinterface

```
interface B {
    default void method() {
        System.out.println("B");
    }
}

interface C extends B {
    @Override
    default void method() {
        System.out.println("C");
    }
}
```

-  C overrides the default method from B.
- This is **explicit** redeclaration with a new body.

◆ 2.4. Declaring a **static** Method – Must Be Redeclared

```
interface A {
    static void utility() {
        System.out.println("A's utility");
    }
}

interface B extends A {
    // Static methods are NOT inherited
    static void utility() {
        System.out.println("B's utility");
    }
}
```

- You must **redeclare** static methods — they are not inherited like instance methods.
- Call them via interface name: `A.utility();`, `B.utility();`

◆ 3. Redeclaring Abstract Methods in Multiple Interfaces (Diamond Shape)

This is important for understanding **redundant abstract declarations**.

```
interface A {
    void method();
}

interface B extends A {
    void method(); //  Redundant, still valid
```

```

}

interface C extends A {
    void method(); // ✅ Also redundant
}

interface D extends B, C {
    // method() is abstract, still just one method to implement
}

```

✅ Java treats all of these as the **same method**, no conflict — unless there's a default involved (see below).

◆ 4. Conflict Resolution: Abstract vs Default

✗ Conflict Example

```

interface A {
    void method();
}

interface B extends A {
    default void method() {
        System.out.println("B");
    }
}

interface C extends A {
    void method(); // abstract
}

interface D extends B, C {
    // ✗ Compilation error unless overridden
}

```

! Java will force you to resolve the conflict between B's default and C's abstract.

✅ Solution:

```

interface D extends B, C {
    @Override
    default void method() {
        B.super.method(); // or your own logic
    }
}

```

◆ 5. What Happens in Implementing Classes?

```

interface A {
    void method();
}

```

```

interface B extends A {
    default void method() {
        System.out.println("B");
    }
}

class MyClass implements B {
    // ✅ Inherits B's default method
}

```

If you do nothing in `MyClass`, `method()` from `B` is used.

If you **override**, your version takes priority.

Key Rules Summary

Scenario	Allowed?	Must Redeclare?	Notes
Interface extends interface with abstract method	✅ Yes	❌ No	Inherited automatically
Redeclare abstract method	✅ Yes	❌ No	Optional, useful for docs
Provide default method	✅ Yes	✅ Yes	Must declare and implement
Provide static method	✅ Yes	✅ Yes	Not inherited — must be called via interface name
Override default in subinterface	✅ Yes	✅ Yes	Must redeclare with <code>default</code> keyword
Conflict between default and abstract	✅ Yes	✅ Yes	Must override to resolve conflict

✅ Final Conceptual Picture

- **Interfaces inherit everything from superinterfaces** (abstract & default).
- **Redeclaration of abstract methods is optional** unless behavior is added.
- **Default methods are only inherited when no conflict arises.**
- **Static methods must be redeclared** and are accessed via interface name.
- **Conflicts must be resolved explicitly**, especially default vs abstract or default vs default.